

Chi tiết các cơ chế bảo vệ nhị phân và Kỹ thuật khai thác

Mục lục

1. NX (No-eXecute) - Cơ chế chống thực thi dữ liệu	2
2. PIE (Position Independent Executable) - Thực thi độc lập vị trí	2
3. Canary (Stack Protector) - "Lính gác" ngăn xếp	3
4. RELRO (Relocation Read-Only) - Bảo vệ bảng liên kết động	4
5. RPATH / RUNPATH - Đường dẫn nạp thư viện	4
6. Symbols (Bảng Ký hiệu)	5
7. FORTIFY_SOURCE - Tối ưu hóa hàm C nguy hiểm	5

1. NX (No-eXecute) - Cơ chế chống thực thi dữ liệu

Lý thuyết chuyên sâu

Trong bộ nhớ của một tiến trình, các dữ liệu được chia thành nhiều phân đoạn (segment) khác nhau như: `.text` (chứa mã lệnh), `.data` / `.bss` (chứa biến toàn cục), Stack (ngăn xếp cục bộ), và Heap (cấp phát động).

Về mặt phần cứng, Bộ Quản lý Bộ nhớ (MMU) của CPU kết hợp với hệ điều hành sử dụng một cờ đặc biệt trong Bảng trang bộ nhớ (Page Table) gọi là NX bit (bit thứ 63 trên kiến trúc x86_64). Khi cờ NX được bật cho một vùng nhớ (ví dụ: Stack hoặc Heap), vùng đó được hệ thống đánh dấu là vùng chỉ có thể Đọc/Ghi (RW) chứ không thể Thực thi (X).

Nếu kẻ tấn công cố tình thay đổi con trỏ lệnh của CPU (thanh ghi RIP/EIP) trở về vùng nhớ có cờ NX, phần cứng CPU sẽ từ chối giải mã các byte tại đó thành lệnh. Thay vào đó, nó ném ra một ngoại lệ phần cứng (Page Fault), và hệ điều hành sẽ ngay lập tức gửi tín hiệu ngắt chương trình bằng lỗi Segmentation Fault (SIGSEGV).

Ví dụ thực tế

Chương trình có một mảng cục bộ `char buf[100]`; trên Stack và dùng hàm `gets(buf)`; (hàm này không kiểm tra độ dài đầu vào, gây lỗi Buffer Overflow).

- **Khi NX Tắt (NO):** Vùng Stack vừa có thể Ghi vừa có thể Thực thi (RWX). Kẻ tấn công sẽ đẩy trực tiếp một đoạn mã máy (Shellcode) vào `buf`, sau đó dùng lỗi tràn bộ đệm để ghi đè Return Address trở ngược lại đúng vào địa chỉ của `buf`. CPU sẽ chạy thẳng đoạn Shellcode đó và cấp quyền điều khiển (Shell).
- **Khi NX Bật (YES):** Vùng Stack chỉ có thể Ghi (RW-). Nếu kẻ tấn công làm y hệt như trên, khi CPU nhảy tới `buf`, nó thấy cờ NX và crash ngay lập tức. Để vượt qua, kẻ tấn công không thể dùng Shellcode tự viết nữa. Thay vào đó, họ phải dùng các công cụ như ROPgadget để tìm các mảnh mã lệnh hợp lệ đã có sẵn trong vùng `.text` (vùng được phép thực thi), sau đó xếp các địa chỉ này lên Stack để gọi lần lượt. Kỹ thuật này gọi là ROP (Return-Oriented Programming).

2. PIE (Position Independent Executable) - Thực thi độc lập vị trí

Lý thuyết chuyên sâu

Thông thường, khi biên dịch một chương trình C, trình biên dịch sẽ gán các địa chỉ vật lý tuyệt đối (hardcoded) cho các hàm và biến. Ví dụ: hàm `main()` luôn ở địa chỉ `0x401000`. Điều này giúp chương trình chạy nhanh nhưng lại rất dễ bị tấn công vì kẻ thù luôn biết chính xác mọi thứ nằm ở đâu.

PIE là cờ yêu cầu trình biên dịch không sử dụng địa chỉ tuyệt đối nữa. Thay vào đó, nó tạo ra các lệnh sử dụng địa chỉ tương đối (ví dụ: "nhảy tới lệnh cách vị trí hiện tại 500 byte"). Khi file nhị phân PIE được chạy, hệ điều hành (thông qua tính năng ASLR) sẽ bốc toàn bộ chương trình và đặt vào một Base_Address (Địa chỉ gốc) hoàn toàn ngẫu nhiên trong bộ nhớ ảo. Mọi địa chỉ của hàm hay biến lúc này sẽ bằng: Base_Address (ngẫu nhiên) + Offset (cố định).

Ví dụ thực tế

Giả sử hàm `admin_shell()` có Offset là `0x1149`.

- **Khi PIE Tắt (NO):** `Base_Address` luôn cố định (thường là `0x400000` trên hệ thống 64-bit). Kẻ tấn công nhắm mắt cũng biết hàm `admin_shell()` nằm ở `0x401149`. Họ chỉ cần đưa con số này vào script (pwntools) để ghi đè con trỏ lệnh.
- **Khi PIE Bật (YES):** Lúc khởi chạy, `Base_Address` có thể là `0x555555554000`, làm hàm `admin_shell()` dời đến `0x555555555149`. Lần chạy sau, nó lại đổi thành dãy số khác. Lúc này, script khai thác bị vô hiệu hóa. Để bẻ khóa, kẻ tấn công buộc phải tìm ra một lỗ hổng khác (ví dụ: Format String) để "đọc trộm" một địa chỉ bất kỳ đang nằm rải rác trên RAM, sau đó làm phép trừ ngược lại để tìm ra `Base_Address` hiện tại, rồi mới tính toán được địa chỉ thật của `admin_shell()`.

3. Canary (Stack Protector) - "Lính gác" ngăn xếp

Lý thuyết chuyên sâu

Canary (bắt nguồn từ việc dùng chim hoàng yến trong hầm mỏ để báo khí độc) là một cơ chế được chèn trực tiếp bởi trình biên dịch (như GCC) vào các hàm có rủi ro tràn bộ đệm.

Khi một hàm bắt đầu thực thi (Function Prologue), nó sẽ lấy một giá trị ngẫu nhiên 8-byte từ bộ nhớ hệ thống (Thread Local Storage - thanh ghi `fs:0x28` trên 64-bit). Nó đặt giá trị này lên Stack, nằm chèn ngay giữa mảng dữ liệu của người dùng và địa chỉ trả về (Return Address). Đặc biệt, byte cuối cùng của Canary thường bị ép thành `0x00` (Null byte) để các hàm in chuỗi của C (như `printf`) dừng lại không in tiếp, tránh bị lộ Canary.

Trước khi hàm kết thúc (Function Epilogue), nó sẽ lấy giá trị Canary trên Stack ra so sánh lại với giá trị gốc trong thanh ghi `fs:0x28`. Nếu khớp, hàm thoát bình thường. Nếu sai lệch dù chỉ 1 bit, chương trình lập tức gọi hàm `__stack_chk_fail()` và tự sát.

Ví dụ thực tế

- **Khi Canary Tắt (NO):** Kẻ tấn công thoải mái nhập 100 chữ 'A' để tràn bộ đệm, ghi đè trón tru từ mảng dữ liệu sang thẳng Return Address.
- **Khi Canary Bật (YES):** Lượng chữ 'A' đó trên đường lan tới Return Address sẽ bắt buộc phải cày nát và thay đổi giá trị của Canary trên Stack. Khi hàm kết thúc, phát hiện Canary đã bị đổi thành chữ 'A', chương trình sẽ in lỗi `*** stack smashing detected ***` và sập. Để tấn công cấu hình này, trước khi ghi đè, bắt buộc phải dùng lỗ hổng để "rò rỉ" (Leak) chính xác 8-byte giá trị của Canary ra ngoài. Sau đó, xây dựng Payload sao cho: chèn dữ liệu rác → chèn đúng giá trị Canary vừa rò rỉ trả lại vị trí cũ → rồi mới chèn Return Address mới.

4. RELRO (Relocation Read-Only) - Bảo vệ bảng liên kết động

Lý thuyết chuyên sâu

Khi một chương trình C gọi các hàm thư viện như `puts()`, `printf()`, nó không chứa sẵn mã lệnh của các hàm này. Thay vào đó, nó tra cứu một bảng nằm trong chính chương trình gọi là GOT (Global Offset Table). Khi chạy lần đầu, địa chỉ thật của hàm `puts` trong bộ nhớ thư viện (libc) sẽ được phân giải và ghi vào GOT. Việc phân giải chậm trễ này gọi là Lazy Binding, yêu cầu bảng GOT phải có quyền Ghi (Writable) trong suốt thời gian chương trình chạy.

Cơ chế RELRO (Full RELRO) thay đổi điều này. Nó buộc hệ điều hành phải phân giải toàn bộ địa chỉ của mọi hàm thư viện ngay tại thời điểm chương trình vừa khởi động (trước khi hàm `main` bắt đầu). Sau khi điền đầy bảng GOT, nó gọi lệnh hệ thống (`mprotect`) để khóa vĩnh viễn bảng GOT thành Chỉ Đọc (Read-Only).

Ví dụ thực tế

Giả sử tồn tại lỗ hổng Arbitrary Write (viết dữ liệu tùy ý vào địa chỉ tùy ý). Chương trình có luồng: gọi `printf(input)` xong rồi gọi `puts("Hello")`.

- **Khi RELRO Tắt (hoặc Partial RELRO):** GOT có quyền ghi. Kẻ tấn công dùng lỗ hổng để chui vào bảng GOT, xóa địa chỉ của hàm `puts` và đề bằng địa chỉ của hàm `system`. Bước tiếp theo, khi chương trình vô tư gọi `puts("Hello")`, nó thực chất đang kích hoạt `system("Hello")`. Đây là kỹ thuật GOT Overwrite khét tiếng.
- **Khi Full RELRO Bật (YES):** Bảng GOT đã bị khóa cứng. Nếu kẻ tấn công cố ghi đè, chương trình sẽ báo lỗi truy cập bộ nhớ và ngắt. Lúc này, mục tiêu tấn công phải chuyển hướng sang các cấu trúc dữ liệu khác không bị khóa (như Return Address trên Stack, hoặc các hook của thư viện cấp phát bộ nhớ).

5. RPATH / RUNPATH - Đường dẫn nạp thư viện

Lý thuyết chuyên sâu

Chương trình thường phụ thuộc vào các thư viện liên kết động (.so trên Linux). Mặc định, Trình liên kết động (Dynamic Linker) của hệ điều hành sẽ tìm các thư viện này ở các đường dẫn an toàn được quy định sẵn (như `/lib/x86_64-linux-gnu` hay `/usr/lib`).

RPATH và RUNPATH là các trường dữ liệu (metadata) được nhúng trực tiếp vào file nhị phân (ELF) lúc biên dịch, nhằm báo cho Linker biết: "Hãy ưu tiên tìm thư viện ở đường dẫn cụ thể này trước". Nếu được cấu hình sai, cơ chế này tạo ra lỗ hổng lớn về logic tải thư viện.

Ví dụ thực tế

Chương trình `admin_tool` được biên dịch với RPATH trỏ tới `/tmp/` (một thư mục lưu rác tạm thời mà bất kỳ người dùng nào cũng có thể đọc/ghi).

- **Khi có RPATH (YES):** Kẻ tấn công (đang có quyền user thấp) tự viết một file mã C độc hại, biên dịch nó thành một thư viện có tên y hệt như thư viện hệ thống (ví dụ: `libc.so.6`) và thả vào thư mục `/tmp/`. Khi một người quản trị (Root) chạy `admin_tool`, chương trình sẽ ưu tiên sục sạo trong `/tmp/` trước, tìm thấy file `libc.so.6` giả mạo và nạp nó vào RAM. Mã độc được thực thi dưới quyền Root (Leo thang đặc quyền - Privilege Escalation).
- **Khi không có RPATH (NO):** Chương trình phốt lờ `/tmp/` và chỉ tìm trong `/lib`, nơi mà kẻ tấn công không có quyền can thiệp.

6. Symbols (Bảng Ký hiệu)

Lý thuyết chuyên sâu

Bảng Ký hiệu (Symbol Table) là khu vực (`.symtab` và `.strtab`) trong file thực thi đóng vai trò như một quyển từ điển. Nó ánh xạ các địa chỉ bộ nhớ không gian thành các tên hàm, tên biến toàn cục bằng ngôn ngữ con người (như `verify_password`, `user_buffer`). Bảng này được thêm vào cốt để phục vụ quá trình Debug lỗi (GDB) của chính lập trình viên.

Việc gỡ bỏ (Strip) bảng ký hiệu hoàn toàn không làm thay đổi các byte lệnh Assembly hay logic của chương trình, CPU vẫn chạy bình thường. Tuy nhiên, nó là rào cản tâm lý và thời gian khổng lồ đối với người dịch ngược.

Ví dụ thực tế

- **Khi Symbols Tắt (YES - Not Stripped):** Đưa file vào các công cụ phân tích (IDA, Ghidra), kẻ tấn công thấy ngay một danh sách rành mạch: `main`, `login`, `check_admin`, `debug_backdoor`. Việc khoanh vùng lỗ hổng diễn ra trong vài phút.
- **Khi Symbols Bật (NO - Stripped):** Bảng ký hiệu đã bị cắt bỏ. Toàn bộ hàm bị đổi thành địa chỉ vật lý, ví dụ: `sub_4011A2`, `sub_4012F0`. Kẻ tấn công phải đi từng dòng Assembly, xem hàm nào gọi system call gì, nhận tham số nào để tự suy luận và đặt lại tên cho từng hàm. Quá trình này đòi hỏi kiến thức sâu và tốn rất nhiều thời gian.

7. FORTIFY_SOURCE - Tối ưu hóa hàm C nguy hiểm

Lý thuyết chuyên sâu

Đây là một tuyến phòng thủ thông minh ở cấp độ Trình biên dịch (Compiler). Rất nhiều hàm thao tác chuỗi và bộ nhớ cơ bản của ngôn ngữ C (như `strcpy`, `sprintf`, `gets`, `memcpy`) không hề có cơ chế tự kiểm tra biên của mảng, dẫn đến lỗi tràn bộ nhớ.

Khi bật cờ FORTIFY trong lúc biên dịch, Compiler sẽ tiến hành phân tích tĩnh (Static Analysis). Nếu nó nhìn thấy một lệnh gọi `strcpy` (đích, nguồn), và nó tính toán được kích thước thực sự của mảng đích tại thời điểm đó, nó sẽ tự động thay thế ngầm hàm `strcpy` bằng một phiên bản siêu an toàn có tên là `__strcpy_chk`. Hàm `chk` (check) này nhận thêm một tham số thứ 3 chính là kích thước tối đa của mảng đích.

Ví dụ thực tế

Đoạn code gốc:

```
char buf[16];  
strcpy(buf, user_input);
```

- **Khi FORTIFY Tắt (NO):** Compiler dịch nguyên bản hàm `strcpy`. Nếu `user_input` dài 50 byte, `strcpy` vẫn cắm đầu copy toàn bộ 50 byte này vào mảng `buf` (chỉ chứa được 16 byte), gây tràn bộ đệm nghiêm trọng.
- **Khi FORTIFY Bật (YES):** Compiler biến đoạn code trên thành: `__strcpy_chk(buf, user_input, 16);`. Lúc chạy, trước khi thực sự copy, hàm này sẽ đo độ dài của `user_input`. Nếu thấy vượt quá con số 16 quy định, nó sẽ hét lên `*** buffer overflow detected ***` và kết liễu chương trình ngay lập tức. Lỗi hổng tràn bộ đệm tại cụm code này bị chặn đứng hoàn toàn từ trong trứng nước.